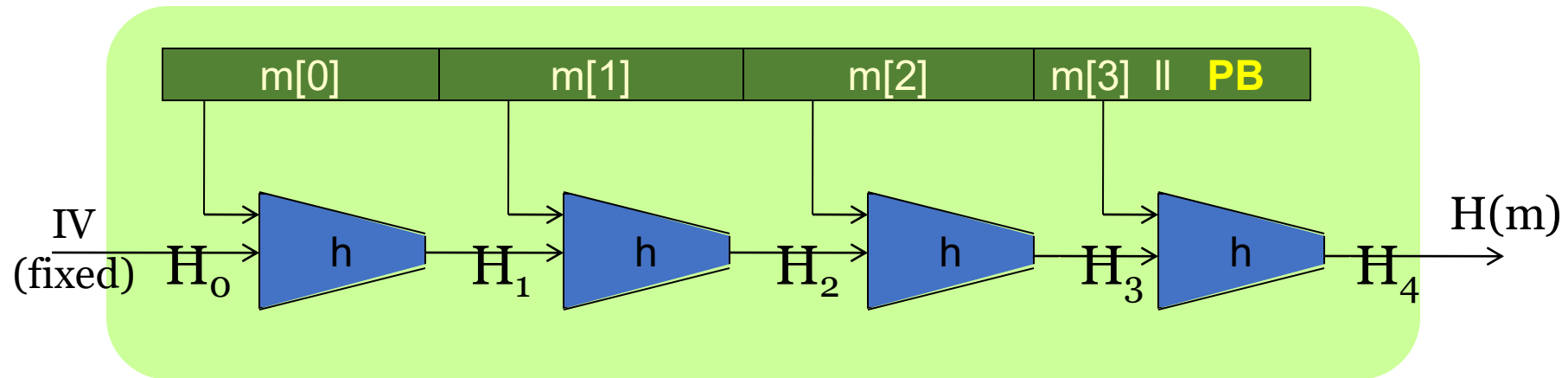


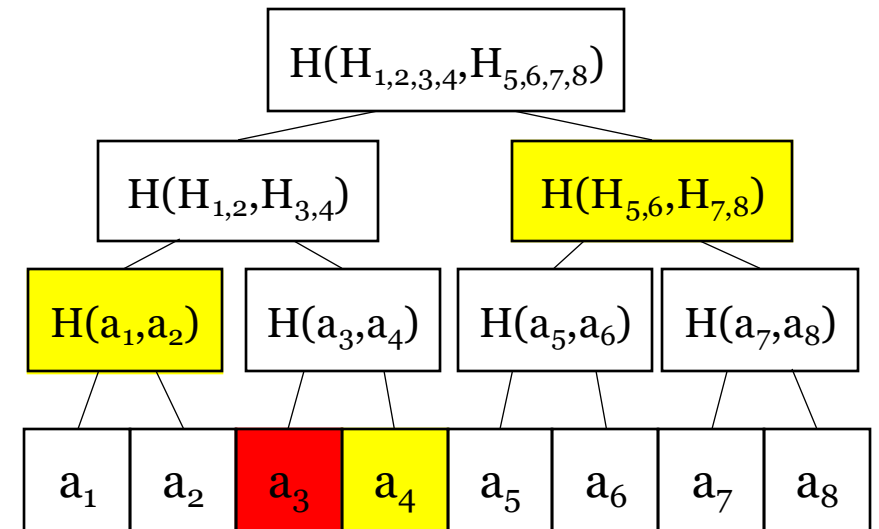
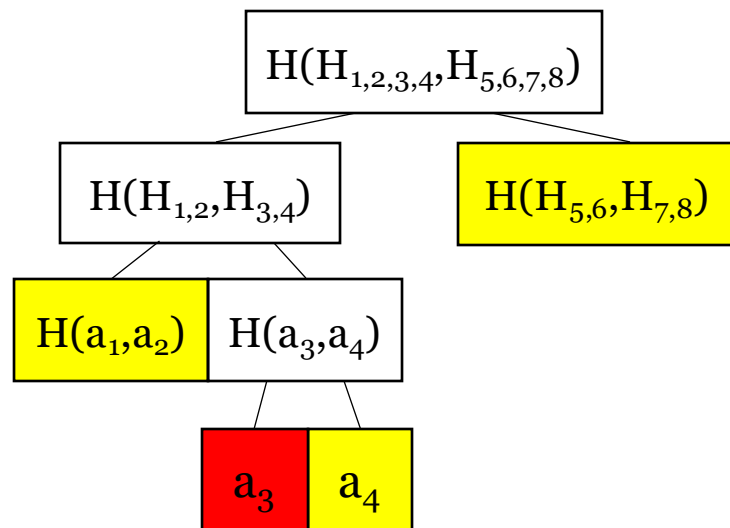
Last time: Merkle-Damgard Transform



PB means the padding bits

Merkle hash tree

- Local storage: $O(1)$
- Server time: $O(\log n)$ with $O(n)$ storage
- Communication: $O(\log n)$
- Verification time: $O(\log n)$
- Update: $O(\log n)$



Message authentication code (MAC)

- $k \leftarrow \text{Gen}(1^n)$
- $t \leftarrow \text{MAC}_k(m)$
- $\{0,1\} \leftarrow \text{Verify}_k(m, t)$

Hash and MAC

- Turn fixed-length MAC to arbitrary length

Let F be a PRF

- $k \leftarrow \text{Gen}(1^n)$
- $\text{MAC}_k(m): t = F_k(m)$
- $\text{Verify}_k(m, t):$ compute $F_k(m)$, check if $t = F_k(m)$

Hash and MAC

MAC_k : unforgeable MAC for n -bit message

H : collision resistant hash function with n -bit output

MAC' : on input $m \in \{0,1\}^*$, compute $t = \text{MAC}_k(H(m))$

Verify' : check if $\text{Verify}_k(H(m), t) = 1$

MAC security

Experiment $\text{MAC}_{A,\Pi}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{MAC}_k(\cdot)$** . Let Q be the set of all queries made by A
3. A outputs (m,t)
4. Experiment outputs 1 if (1) $\text{verify}_k(m, t)=1$ and (2) $m \notin Q$

Definition: a Mac $\Pi = (\text{Gen}, \text{MAC}, \text{Verify})$ is **unforgeable/secure**, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{MAC}_{A,\Pi}(n) = 1] \leq \text{negl}(n)$$

Proof sketch

Suppose A forges t^* for $m^* \notin Q$ with a non-negligible prob.

- Case 1: there is $m \in Q$ such that $H(m) = H(m^*)$, then collision, contradiction.
- Case 2: for all $m \in Q$, $H(m) \neq H(m^*)$, then define $Q_H = \{H(m) | m \in Q\}$, $H(m^*) \notin Q_H$ is a forgery of the fixed-length MAC. Contradiction.

Using hash for MAC

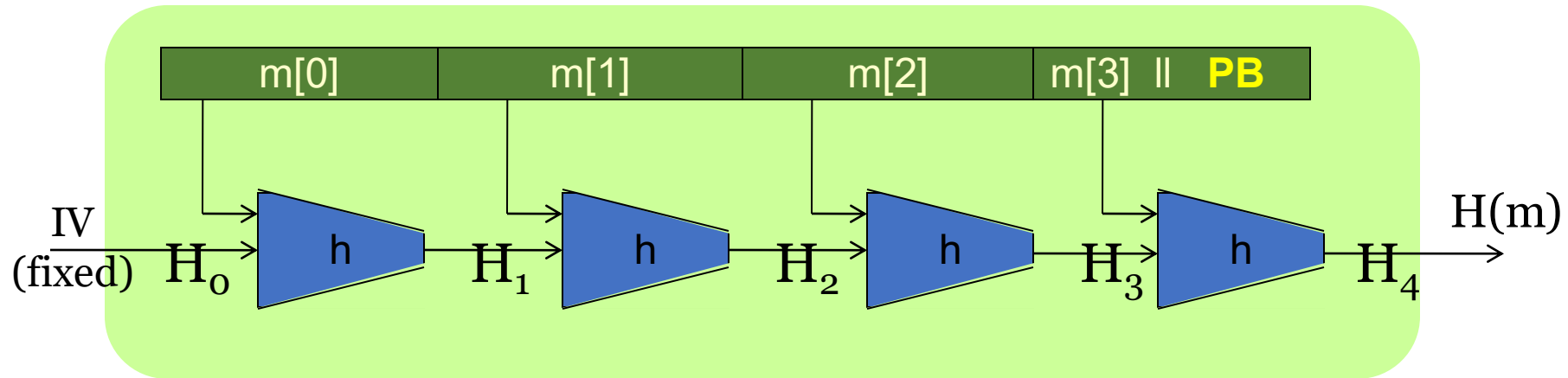
$$\text{MAC}_k(m): t = H(k||m)$$

Length-extension attack

- $t = H(k||m)$ is vulnerable to Length-extension attack
- Worked because we can extend a Merkle-Damgård hash w/o knowing key at start

Merkle-Damgard Transform

Dealing with arbitrary length input



Construction: (Gen, h) a collision resistant hash function $\{0,1\}^{n+n'} \rightarrow \{0,1\}^n$

Gen: unchanged

H : for input string $x \in \{0,1\}^*$ of length $L < 2^{n'}$

1. Padding: append 10000... concatenated with $L = |x|$ such that the result is a multiple of n'
2. $y_0 = IV = 0^n$
3. For $i = 1, 2, \dots, t$, $y_i = h(y_{i-1} || x_i)$

HMAC

- $\text{HMAC}_k(m) = H\left((k \oplus \text{opad}) || H((k \oplus \text{ipad}) || m)\right)$

Idea of HMAC

$\text{MAC}_k(m): t = H(k || m)$ secure for fixed length m

Hash and MAC:

$$\text{MAC}_k(m) = H(k || H(m))$$

Idea of HMAC

Hash and MAC:

$$\text{MAC}_k(m) = H(k || H(m))$$

Weak collision resistant of H :

$$\text{MAC}_k(m) = H(k_1 || H(k_2 || m))$$

HMAC

- $\text{MAC}_k(m) = H(k_1 || H(k_2 || m))$

- $\text{HMAC}_k(m) = H((k \oplus \text{opad}) || H((k \oplus \text{ipad}) || m))$

Opad (outer padding) and ipad (inner padding) are public constant

How does it all fit together?

	Symmetric		Asymmetric
Confidentiality Non-deterministic	Symmetric key encryption	Key	Public key encryption
Integrity deterministic	Message authentication code		Digital signature
	Hash function	No key	

Padding oracle attack and CPA security

- Change ciphertext $c_0 \oplus m_1 = c_0' \oplus m_1' = f_K^{-1}(c_1)$
- CPA Experiment
 1. A key k is generated using $\text{Gen}(1^n)$
 2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** . A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
 3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_k(m_b)$ is given to A .
 4. A continues to have **oracle access to $\text{Enc}_k(\cdot)$** and outputs b'
 5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Decryption oracle

- A is given oracle access to **Dec_k(·)**
 - Query any c , get $m = \text{Dec}_k(c)$
- More powerful than the padding oracle
 - Padding oracle: query any c , get $P_k(c) \rightarrow 0$ or 1

Attempt for a stronger definition

- Experiment

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** and **$\text{Dec}_k(\cdot)$** . A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_k(m_b)$ is given to A .
4. A continues to have **oracle access to $\text{Enc}_k(\cdot)$** and **$\text{Dec}_k(\cdot)$** , and outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Chosen Ciphertext Attack (CCA) security

Experiment $\text{PrivK}_{A,\Pi}^{\text{CCA}}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** and **$\text{Dec}_k(\cdot)$** . A outputs two messages $m_0, m_1 \in \mathcal{M}$ s.t. $|m_0| = |m_1|$
3. A random bit $b \in \{0,1\}$ is chosen. $c = \text{Enc}_k(m_b)$ is given to A .
4. A continues to have **oracle access to $\text{Enc}_k(\cdot)$** and **$\text{Dec}_k(\cdot)$** , but is not allowed to **query $\text{Dec}_k(c)$** . A outputs b'
5. Experiment outputs 1 if $b=b'$, and 0 otherwise.

Definition: a private-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is CCA-secure, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{PrivK}_{A,\Pi}^{\text{CCA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$$

Achieving CCA security

- Idea: make the decryption oracle useless
- Integrity check on the ciphertext
- Intuition: the attacker can't change the ciphertext on his own!
 - Any query to $\text{Dec}_k(\cdot)$ must be from $\text{Enc}_k(\cdot)$
 - The attacker already knows the message

Unforgeable Encryption

Experiment $\text{EncForge}_{A,\Pi}(n)$:

1. A key k is generated using $\text{Gen}(1^n)$
2. Adversary A is given input 1^n , and **oracle access to $\text{Enc}_k(\cdot)$** .
3. A outputs a ciphertext c . Let $m = \text{Dec}_k(c)$, and Q denote the set of all queries made by A
4. Experiment outputs 1 if (1) $m \neq \perp$ and (2) $m \notin Q$.

Definition: a private-key encryption $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ is **unforgeable**, if for all probabilistic polynomial time adversaries A , there is a negligible function negl such that for all n ,

$$\Pr[\text{EncForge}_{A,\Pi}(n) = 1] \leq \text{negl}(n)$$

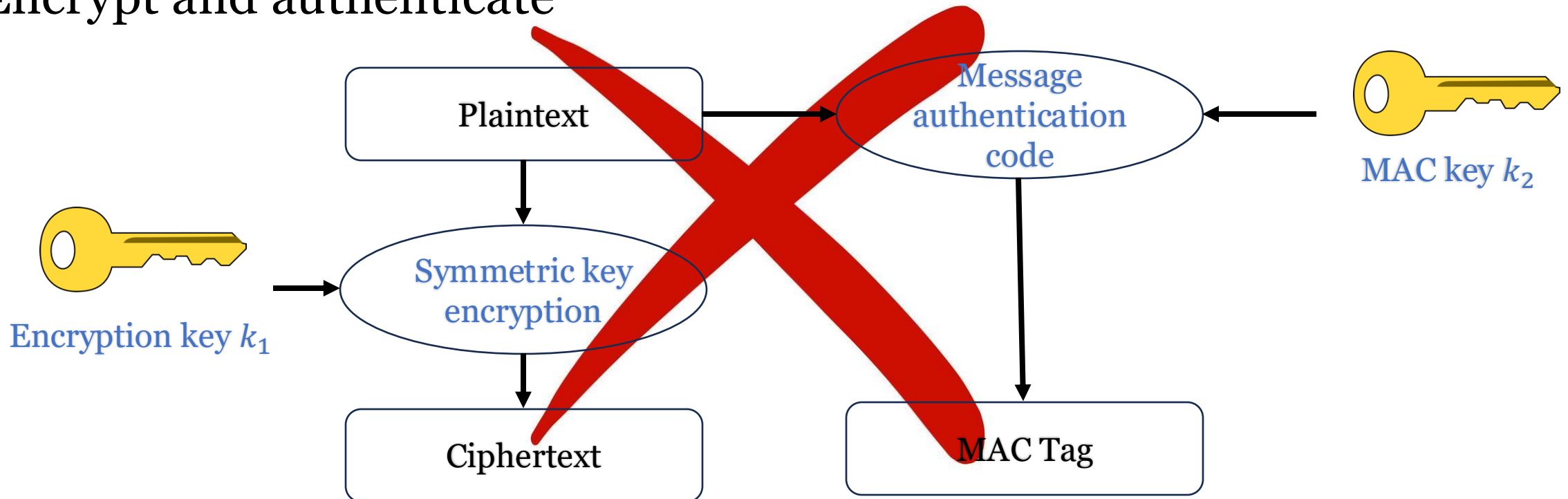
Authenticated Encryption

- Definition: a private-key encryption scheme is an authenticated encryption if it is **CCA-secure** and **unforgeable**.

Generic constructions

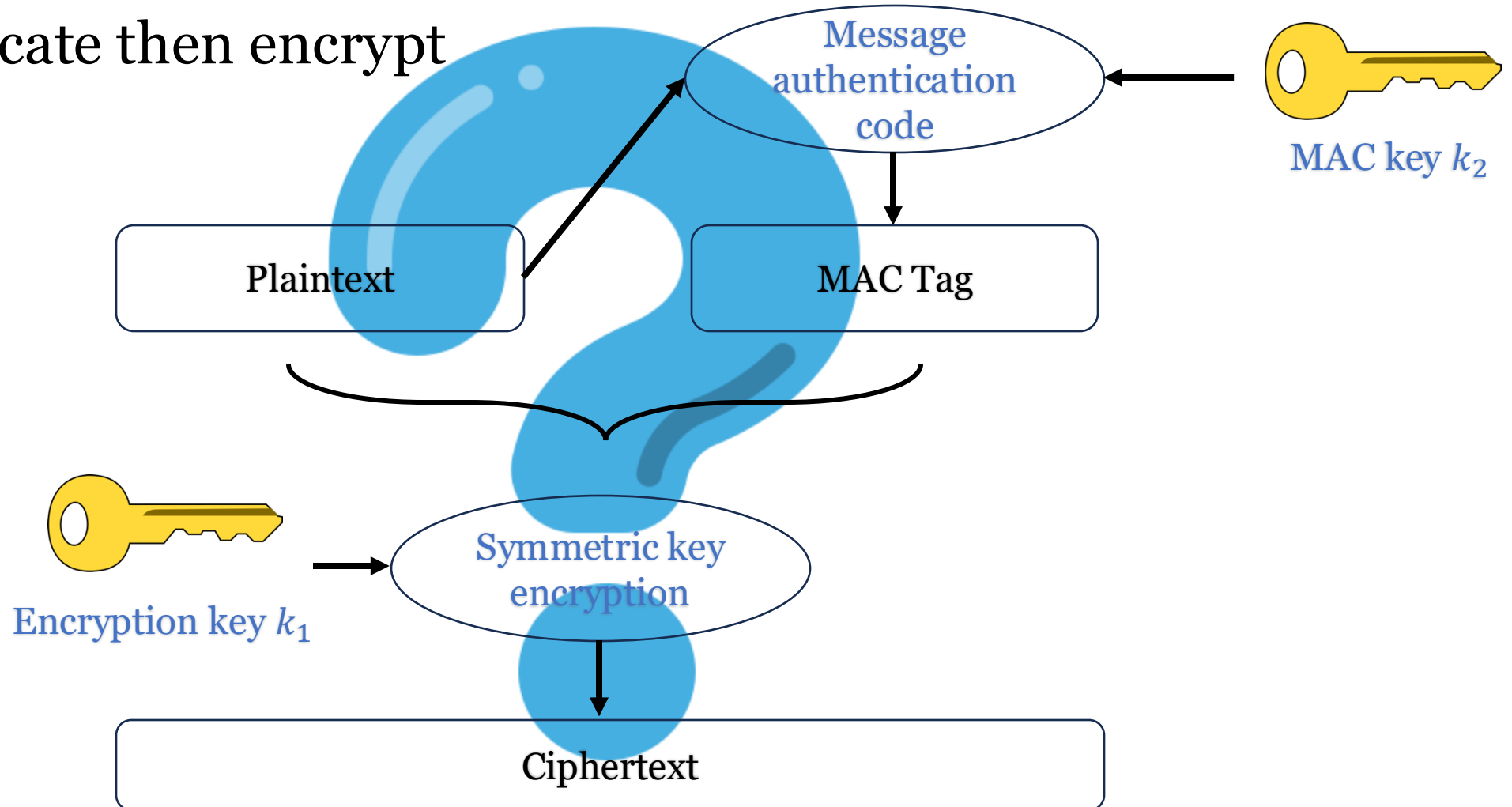
- CPA-secure encryption scheme
- Strongly unforgeable MAC

1. Encrypt and authenticate



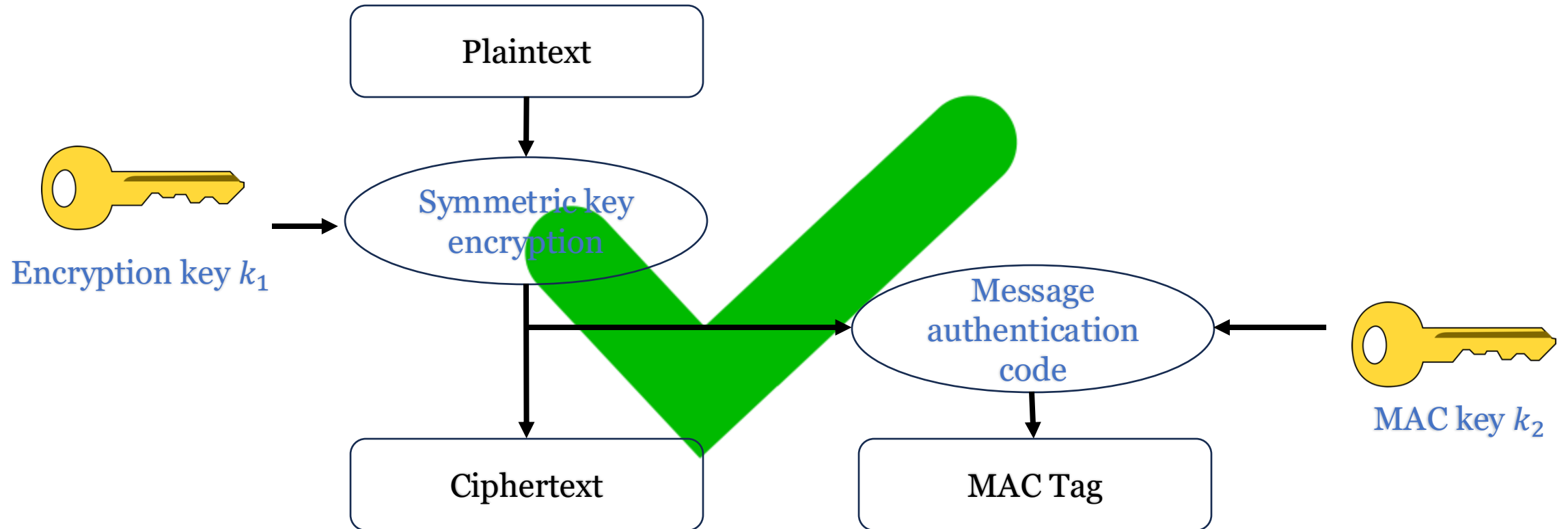
Generic constructions

2. Authenticate then encrypt



Generic constructions

3. Encrypt then authenticate



Encrypt then authenticate

- $k_1, k_2 \leftarrow \text{Gen}(1^n)$
- $\text{Auth_Enc} : c \leftarrow \text{Enc}_{k_1}(m), t = \text{MAC}_{k_2}(c)$, output c, t
- $\text{Auth_Dec} : \text{verify}_{k_2}(c, t) = 1$. If yes, $m = \text{Dec}_{k_1}(c)$. Otherwise, output $\perp$

Theorem: if the encryption is **CPA-secure**, and the MAC is **strongly unforgeable**, the construction is an **authenticated encryption**.

Proof sketch:

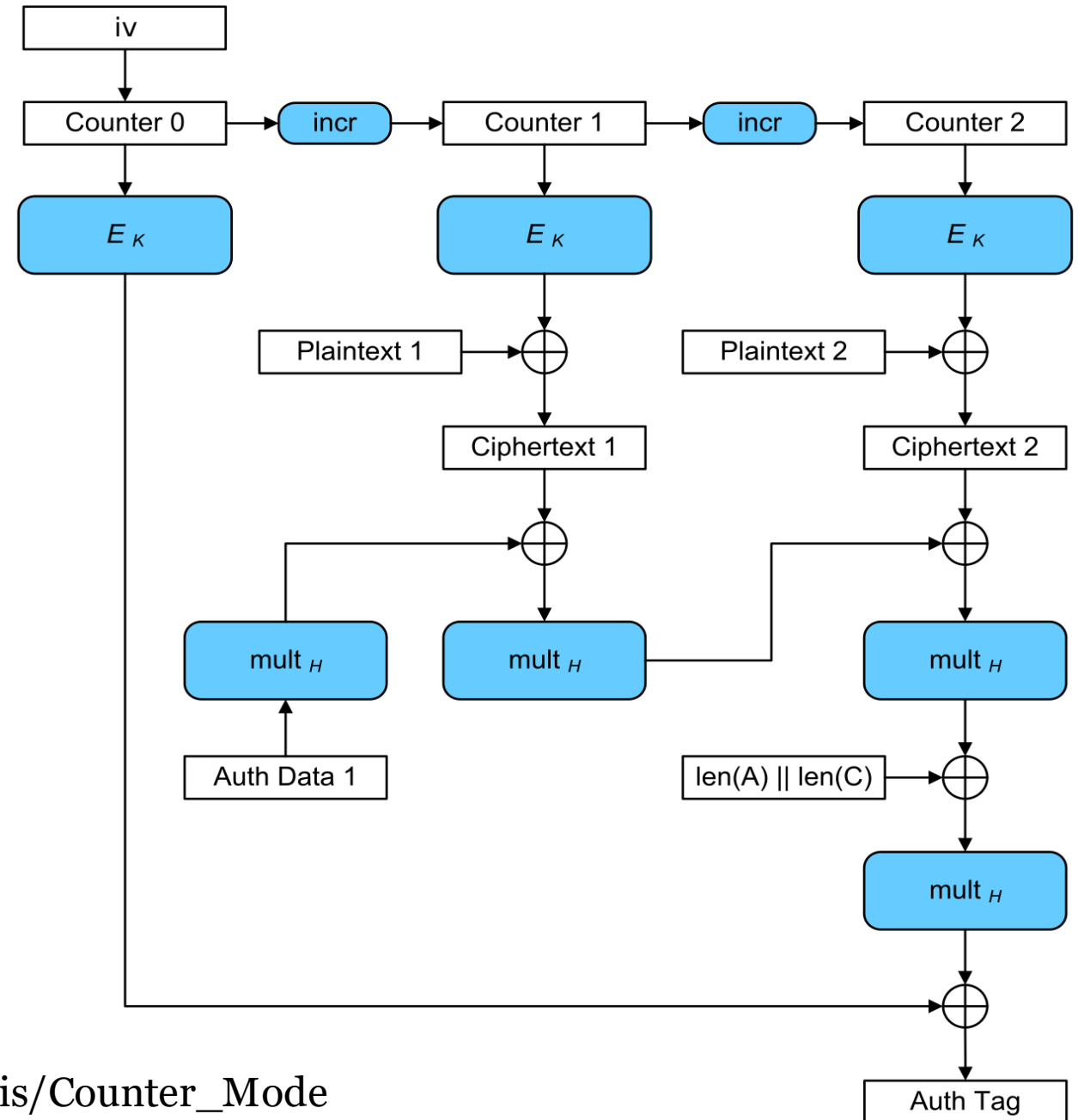
1. Encryption is unforgeable
2. Decryption oracle is useless in CCA experiment

Standardized schemes

GCM (Galois/counter mode)
CTR mode + GMAC

Encrypt then authenticate

- Same key
- Same IV



Standardized schemes

- CCM (CTR with CBC-MAC)

Authenticate then encrypt, **but can be proven secure!**

Same key

CCM

